UICT — Uganda
Department of Computer Science

CSC 122

Structured Software Development

C Programming

Ground to Mastery Study Notes

Bravehart
DCS Year 1, Semester 2
Academic Year 2025/2026

Assessment: Coursework 40% | Final Exam 60%

Assessment Breakdown

Component	Weight	Details
Continuous Assessment (Coursework)	**40%**	Tests, assignments, practicals
Final Examination	**60%**	Written final exam at semester end

Minimum pass mark: 50% overall

You must pass BOTH components to pass the course unit.

Unit 1: Introduction to C Programming

C is a general-purpose, procedural programming language developed by Dennis Ritchie at Bell Labs in 1972. It remains one of the most widely used languages due to its efficiency, portability, and low-level memory access capabilities.

Key Characteristics of C

- Compiled language — code is translated to machine code before execution
- Strongly typed — variables must be declared with a data type
- Procedural — programs are built from functions (procedures)
- Portable — C programs can run on different hardware with minimal changes
- Low-level access — allows direct manipulation of memory using pointers

Basic Program Structure

Every C program must have a main() function. The #include directive imports header files. The printf() function outputs text to the screen. All statements end with a semicolon (;).

Term	Definition
Compiler	A program that translates C source code into executable machine code
Header file	A file (e.g., stdio.h) containing function declarations and macros
main()	The entry point of every C program; execution begins here
printf()	Standard library function used to display output on the screen
scanf()	Standard library function used to read input from the keyboard
#include	Preprocessor directive to include external header files
return 0	Signals successful program termination to the operating system

> EXAM TIP: Always remember that C is case-sensitive. 'Main' and 'main' are different. The main() function must always return an integer (return 0 means success). Include stdio.h whenever you use printf or scanf.

Unit 2: Data Types, Variables & Operators

Variables are named storage locations in memory. Every variable must be declared with a data type before use. The data type tells the compiler how much memory to allocate and how to interpret the stored value.

Data Type	Size	Range / Use
int	4 bytes	Whole numbers: -2,147,483,648 to 2,147,483,647
float	4 bytes	Decimal numbers with ~6-7 significant digits
double	8 bytes	Decimal numbers with ~15 significant digits (more precise)
char	1 byte	Single character, e.g., 'A', 'z', '5'
void	0 bytes	Represents absence of a type (used in functions)

Arithmetic Operators

- + (Addition): adds two values
- - (Subtraction): subtracts right from left
- * (Multiplication): multiplies two values
- / (Division): divides; integer division truncates decimal part
- % (Modulus): gives the remainder after integer division

Relational & Logical Operators

- == (Equal to), != (Not equal)
- > (Greater than), < (Less than), >= , <=
- && (Logical AND): true only if BOTH conditions are true
- || (Logical OR): true if AT LEAST ONE condition is true
- ! (Logical NOT): reverses the truth value

> EXAM TIP: Do not confuse = (assignment) with == (comparison). Using = inside an if() condition is a common bug. Also note integer division: 7/2 gives 3, not 3.5 — use float or double for decimal results.

Unit 3: Control Flow — Decision Making

Control flow statements allow the program to make decisions and execute different blocks of code based on conditions. This makes programs dynamic and responsive to data.

if / else if / else Statement

The if statement evaluates a condition. If true, the if-block executes. If false, the else-block executes. The else if allows checking multiple conditions in sequence.

switch Statement

The switch statement is used when a variable is compared against multiple constant values. It is more readable than many else-if chains. Each case must end with break to prevent fall-through.

- switch evaluates an expression and jumps to the matching case
- break prevents execution from falling into the next case
- default executes when no case matches — similar to else
- switch works only with int, char, and enum types — not float or string

Keyword	Purpose
if	Executes a block when a condition is true
else	Executes a block when the if condition is false
else if	Tests additional conditions after the first if fails
switch	Selects one of many code blocks based on a variable's value
case	Represents one possible value in a switch statement
break	Exits the switch or loop immediately
default	Handles any value not matched by a case label

> EXAM TIP: A missing break in switch causes fall-through — execution continues into the next case even without a match. This is a common exam question. Also remember: switch cannot test ranges (e.g., x > 5); use if-else for that.

Unit 4: Loops & Iteration

Loops allow a block of code to execute repeatedly. C provides three types of loops, each suited to different situations.

Loop Type	When to Use
for loop	When the number of iterations is known in advance
while loop	When the condition is checked before each iteration (may not run at all)
do-while loop	When the loop body must execute at least once before checking condition

Loop Control Keywords

- break — exits the loop immediately, skipping remaining iterations
- continue — skips the current iteration and moves to the next
- Nested loops — a loop inside another loop; common for 2D patterns and matrices
- Infinite loop — occurs when the condition never becomes false; use break to escape

for Loop Structure

A for loop has three parts: initialisation (runs once), condition (checked before each iteration), and update (runs after each iteration). Example: for(int i=0; i<10; i++) repeats 10 times (i from 0 to 9).

> EXAM TIP: Know the difference — while checks condition BEFORE the body runs. do-while checks AFTER, so the body always runs at least once. Be careful with off-by-one errors: for(i=1; i<=10; i++) runs 10 times; for(i=0; i<10; i++) also runs 10 times.

Unit 5: Functions

A function is a self-contained block of code that performs a specific task. Functions promote code reuse, modularity, and readability. Every C program has at least one function: main().

Function Components

- Return type — the data type of the value the function returns (or void if none)
- Function name — a unique identifier following variable naming rules
- Parameters — input values passed to the function (optional)
- Function body — the block of code that executes when the function is called
- Return statement — sends a value back to the calling code

Passing Arguments

Method	Description	Changes original?
Pass by Value	A copy of the variable is passed; changes inside function do not affect the original	No
Pass by Reference	The address (pointer) is passed; changes inside function affect the original variable	Yes

Scope of Variables

- Local variables — declared inside a function; only accessible within that function
- Global variables — declared outside all functions; accessible from anywhere in the file
- Static variables — retain their value between function calls

> EXAM TIP: Declare functions before calling them (use prototypes). A function prototype tells the compiler the return type and parameters before the full definition appears. Prototype format: returnType functionName(paramType);

Unit 6: Arrays & Strings

An array is a collection of elements of the same data type stored in contiguous memory locations. Arrays allow efficient storage and manipulation of multiple related values using a single variable name.

Term	Definition
Array	A fixed-size collection of same-type elements accessed by an index
Index	The position of an element in an array; always starts at 0 in C
String	An array of characters terminated by the null character '\0'
strlen()	Standard library function that returns the length of a string
strcpy()	Copies one string into another
strcmp()	Compares two strings; returns 0 if equal
strcat()	Concatenates (joins) two strings together

Key Rules for Arrays

- Array indices start at 0 — an array of size 5 has indices 0 to 4
- Accessing an index out of bounds causes undefined behaviour (a dangerous bug)
- Array size must be a constant or known at compile time (in standard C)
- Multidimensional arrays (e.g., int matrix[3][4]) store data in rows and columns
- Strings in C must always have space for the null terminator '\0'

> EXAM TIP: Remember that strings in C are arrays of char ending with '\0'. The null terminator is automatically added by string literals ("hello" has 6 characters including '\0'). Use string functions from string.h, not direct assignment with = for strings.

Unit 7: Pointers & Memory Management

A pointer is a variable that stores the memory address of another variable. Pointers are one of C's most powerful features, enabling dynamic memory allocation, efficient array handling, and pass-by-reference.

Term	Definition
Pointer	A variable that holds the memory address of another variable
& operator	Address-of operator; returns the memory address of a variable
* operator	Dereference operator; accesses the value at the address stored in a pointer
NULL pointer	A pointer that points to nothing (address 0); used to indicate no valid address
malloc()	Allocates a specified number of bytes of memory dynamically
free()	Releases dynamically allocated memory back to the system
Memory leak	Occurs when allocated memory is not freed, wasting system resources

Pointer Rules

- Always initialise pointers before use — an uninitialised pointer causes undefined behaviour
- Always free dynamically allocated memory when done to prevent memory leaks
- A pointer and the variable it points to must have the same base type
- Pointer arithmetic: incrementing a pointer moves it forward by the size of its data type

> EXAM TIP: Know the difference between * in declaration (int *p declares a pointer) versus * in expression (*p dereferences, giving the value at the address). Also: int *p = &x means 'p holds the address of x'.

Unit 8: Structures & File Handling

A structure (struct) is a user-defined data type that groups variables of different types under one name. File handling allows programs to read from and write to files, enabling data persistence beyond program execution.

Structures (struct)

- Use the struct keyword to define a new composite data type
- Members (fields) can be of different data types
- Access members with dot notation: student.name or arrow notation for pointers: ptr->name
- typedef can create an alias for a struct to simplify declarations

File Handling Functions

Function	Purpose
fopen()	Opens a file; returns a FILE pointer; NULL if file cannot be opened
fclose()	Closes an open file and flushes any buffered output
fprintf()	Writes formatted text to a file (like printf but to a file)
fscanf()	Reads formatted data from a file (like scanf but from a file)
fgets()	Reads a line of text from a file safely
feof()	Returns true when the end of a file has been reached

File Opening Modes

- "r" — read only; file must exist
- "w" — write only; creates new or overwrites existing file
- "a" — append; adds to end of existing file
- "r+" — read and write; file must exist
- "w+" — read and write; creates new or overwrites

> EXAM TIP: Always check if fopen() returns NULL before using the file pointer. Always call fclose() when done. Forgetting to close files can cause data loss or corruption.

Unit 9: Structured Programming Principles & Algorithms

Structured programming is a programming paradigm that emphasises the use of three control structures: sequence, selection, and iteration. It discourages the use of unstructured jumps (like goto) to improve readability and maintainability.

Term	Definition
Algorithm	A step-by-step procedure to solve a problem
Pseudocode	An informal, human-readable description of an algorithm using plain language
Flowchart	A visual diagram using standard symbols to represent an algorithm
Sequence	Statements execute one after another in order
Selection	A decision point where one of several paths is chosen (if, switch)
Iteration	A block that repeats (for, while, do-while)
Top-down design	Breaking a large problem into smaller subproblems (functions)
Modularisation	Dividing a program into independent, manageable modules (functions)

Software Development Life Cycle (SDLC)

- 1. Problem Analysis — understand and define the problem clearly
- 2. Algorithm Design — create pseudocode or flowchart
- 3. Coding — translate the algorithm into C
- 4. Compilation — convert source code to executable
- 5. Testing & Debugging — find and fix errors
- 6. Documentation — write comments and user guides
- 7. Maintenance — update and improve the program over time

> EXAM TIP: Know the three types of programming errors: Syntax errors (grammar mistakes caught by compiler), Logic errors (program runs but gives wrong output), and Runtime errors (crashes during execution, e.g., division by zero, NULL pointer dereference).